

Treinamento em Segurança Ofensiva 2024/2025

Relatório de Atividades	
Módulo 4:	Pentest em Aplicações Web
Atividade 2:	Vulnerabilidades de Cross-Site Scripting (XSS)
Nome:	Luiz Eduardo de Andrade

1. Introdução

Estas atividades fazem parte do módulo *Pentest em Aplicações Web*, para a qual utilizamos a sala do PortSwigger - <https://portswigger.net/>

No contexto desta sala, nosso foco é:

- 1) Através de vulnerabilidade de cross scripting no campo de pesquisa conseguir emitir um alerta na página do blog;
- 2) Conseguir através de um script em uma página de comentários, o nome que consta no comentário vira um link para uma mensagem de alerta.
- 3) Mostrar uma vulnerabilidade DOM refletida para que os dados na resposta sejam ecoados na saída através de uma injeção que gera um alerta;
- 4) Utilizar o armazenamento DOM na funcionalidade de comentário do blog, quando o comentário é enviado uma mensagem de alerta aparece na tela.

A sala do Portswigger é composta por quatro tarefas:

- 1) XSS refletido em atributo com colchetes codificados em HTML
- <https://portswigger.net/web-security/cross-site-scripting/contexts/lab-attribute-angle-brackets-html-encoded>
- 2) XSS armazenado no atributo href âncora com aspas duplas codificado em HTML
- <https://portswigger.net/web-security/cross-site-scripting/contexts/lab-href-attribute-double-quotes-html-encoded>
- 3) DOM XSS refletido - <https://portswigger.net/web-security/cross-site-scripting/dom-based/lab-dom-xss-reflected>
- 4) DOM XSS armazenado - <https://portswigger.net/web-security/cross-site-scripting/dom-based/lab-dom-xss-stored>

2. Desenvolvimento

2.1. Configuração do ambiente

Os seguintes passos foram necessários para configurar o ambiente necessário para o desenvolvimento das atividades:

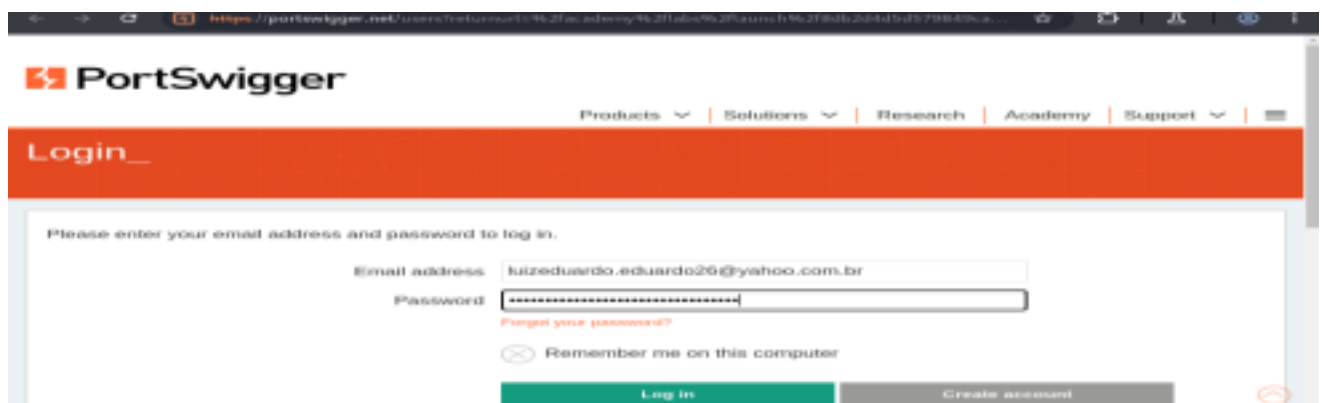
- 1) Acessar os sites informados acima.
- 2) Utilizar a VM do Kali Linux: Para utilizar o Burp e executar alguns comandos no terminal.

2.2. Atividade prática

O objetivo desta atividade é realizar as 4 tarefas propostas no PortSwigger

2.3. Atividade 1 - XSS refletido em atributo com colchetes codificados em HTML.

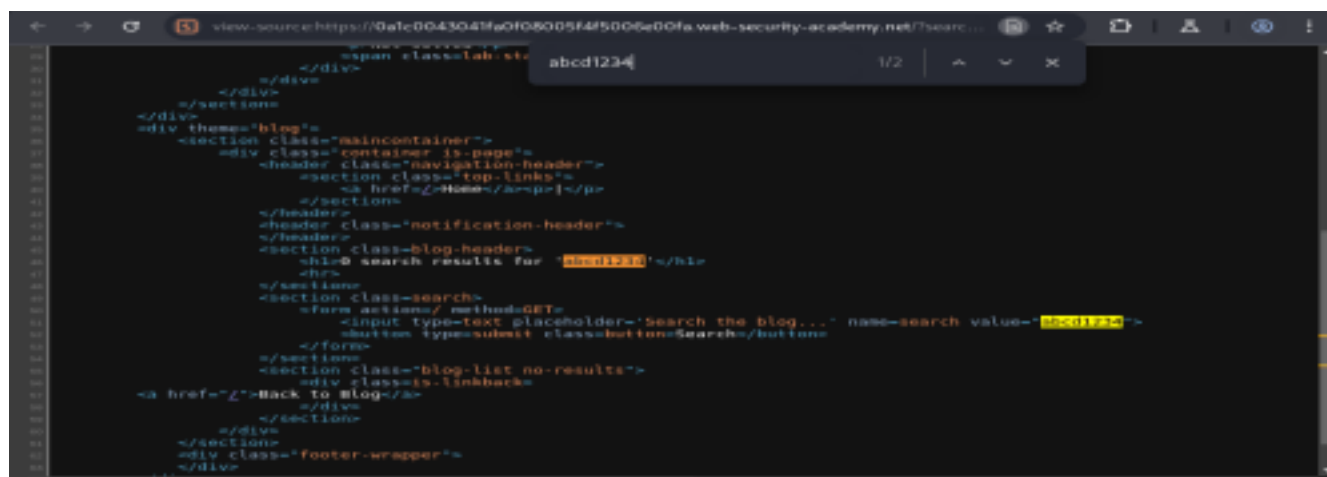
Passo 1 – Entre com seu login e senha no PortSwigger no site: <https://portswigger.net/web-security/cross-site-scripting/contexts/lab-attribute-angle-brackets-html-encoded> e clique em acesse o laboratório



Passo 2 – Vai aparecer o site: <https://0a1c0043041fa0f08005f4f5006e00fa.web-security-academy.net/> e procure no campo search por abcd 1234



Passo 3 – Clique com o botão direito e selecione view source page, depois procure por abcd1234 pressionando o control + F

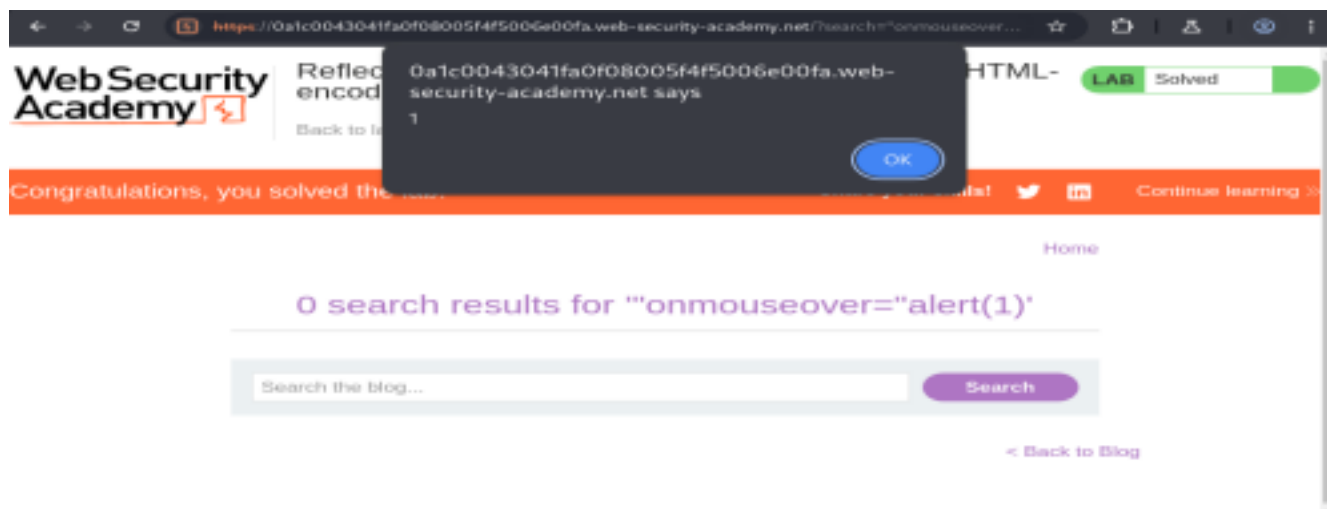


Passo 4 – Depois no campo de busca digite o comando “onmouseover=”alert(1) este comando xss gera um alerta.



3

Passo 5 – O laboratório foi solucionado com a mensagem de alerta 1.

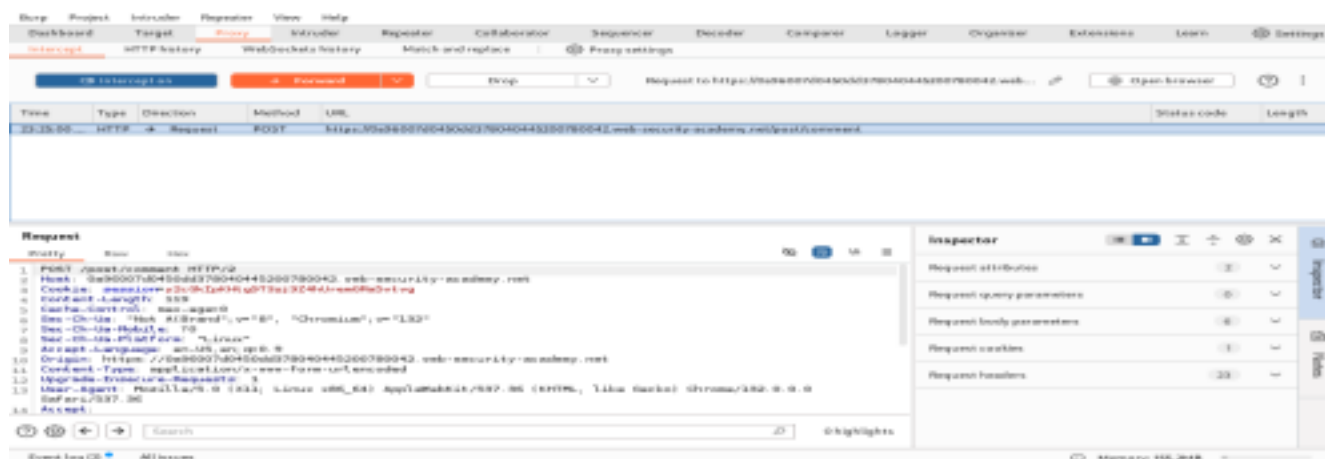


2.4. Atividade 2 – XSS armazenado no atributo href âncora com aspas duplas codificado em HTML.

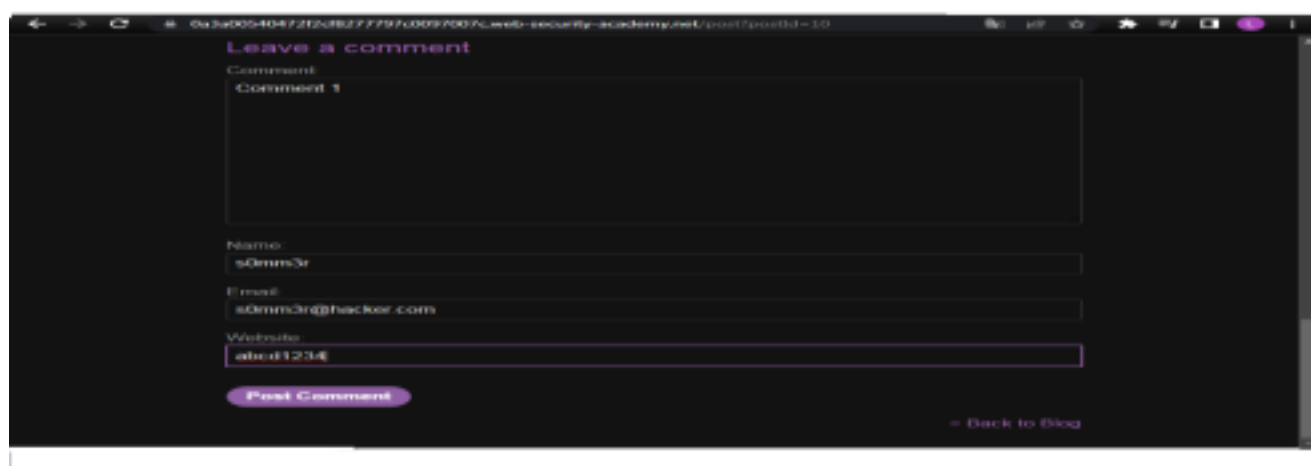
Passo 1 – Vai aparecer o site: <https://0a96007d0450dd378040445200780042.web-security-academy.net/> e clique em view post.



Passo 2 – Acesse o Burp e clique em forward para que apareça a página de comentários.



Passo 3 – Escreva seu comentário e clique em Post Comment

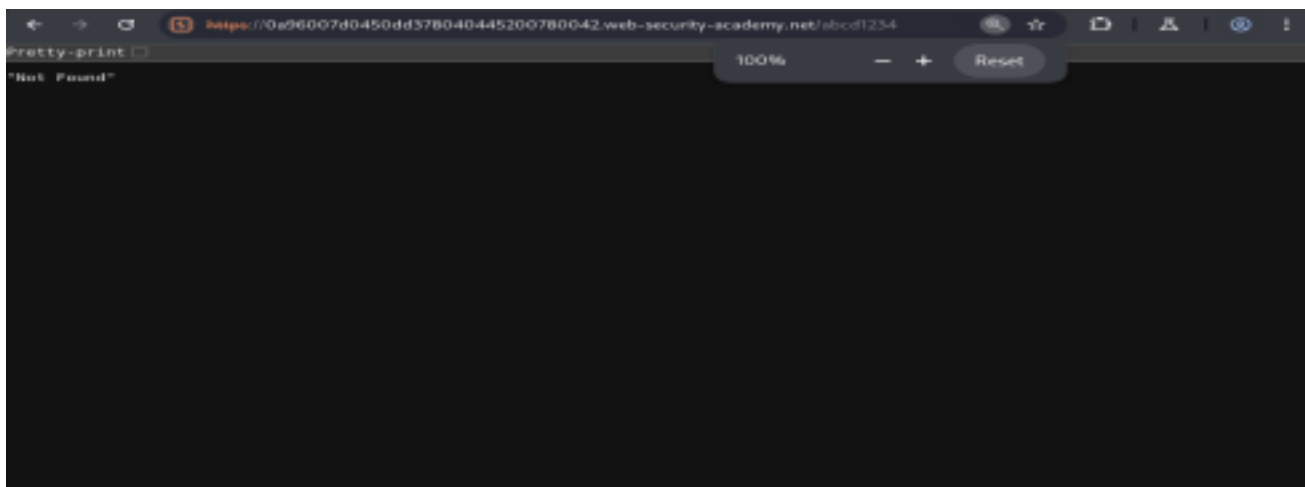


Passo 4 – Aparecerá que seu comentário foi realizado.

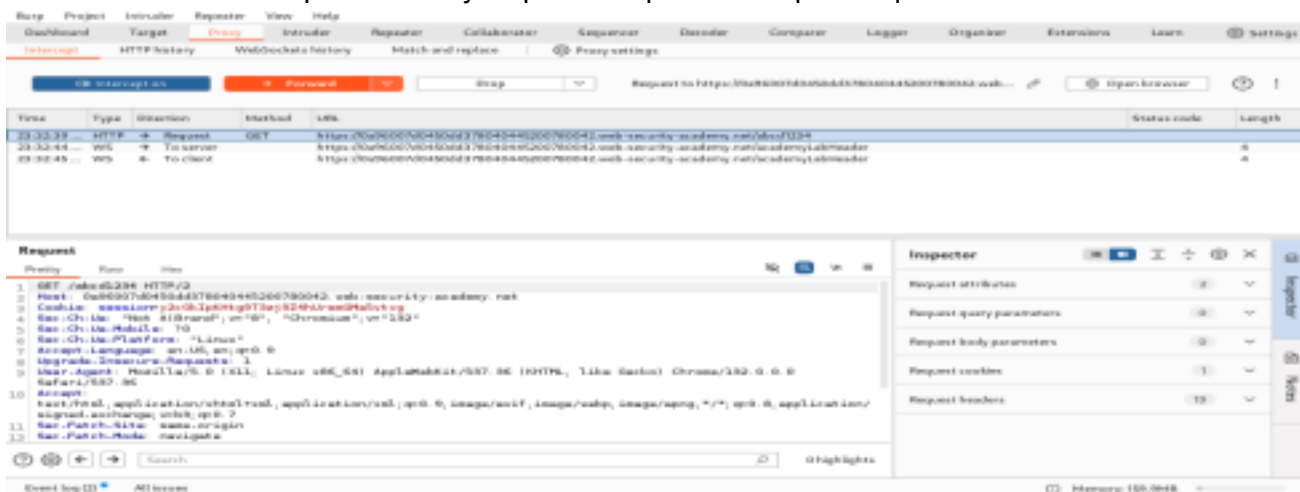


5

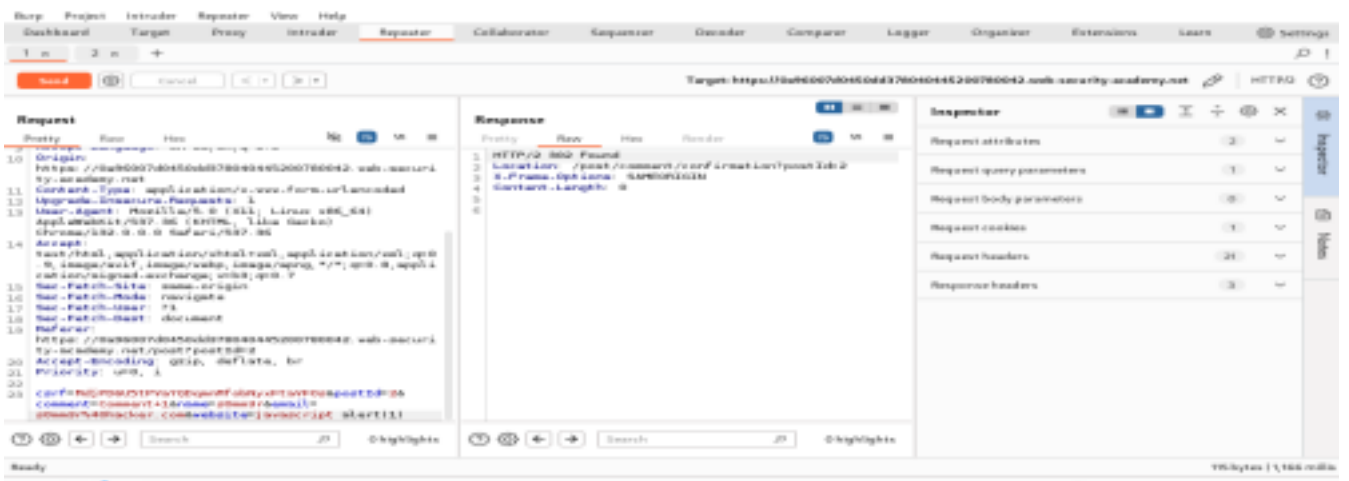
Passo 5 – Tente no browser verificar se depois do academy.net coloque /abcd1234 mas aparecerá que não foi encontrado.



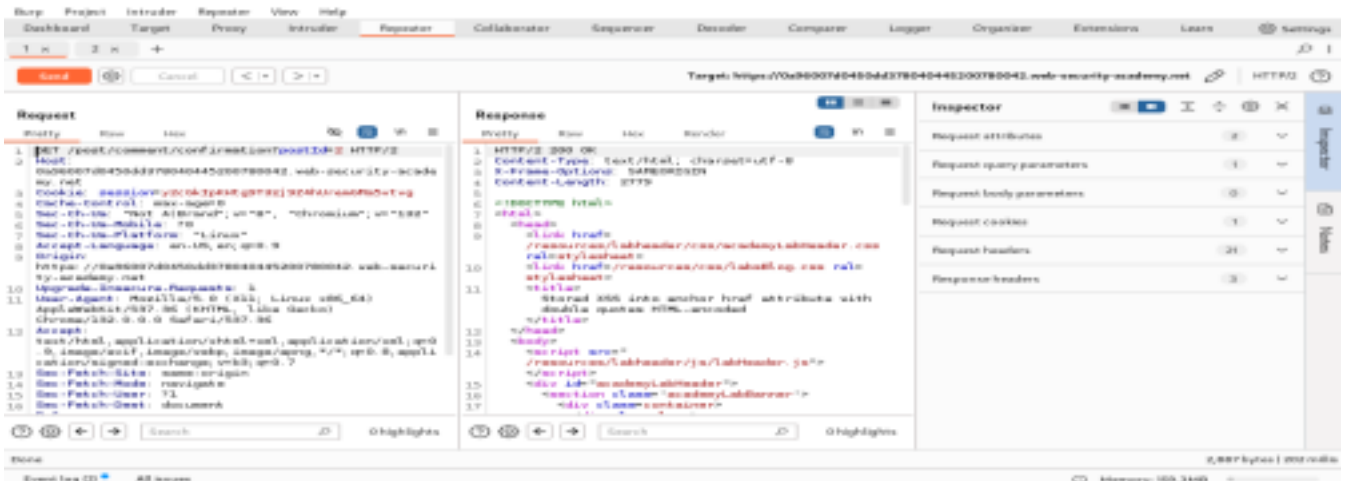
Passo 6 – Volte ao Burp Community clique em repeater e depois clique em forward



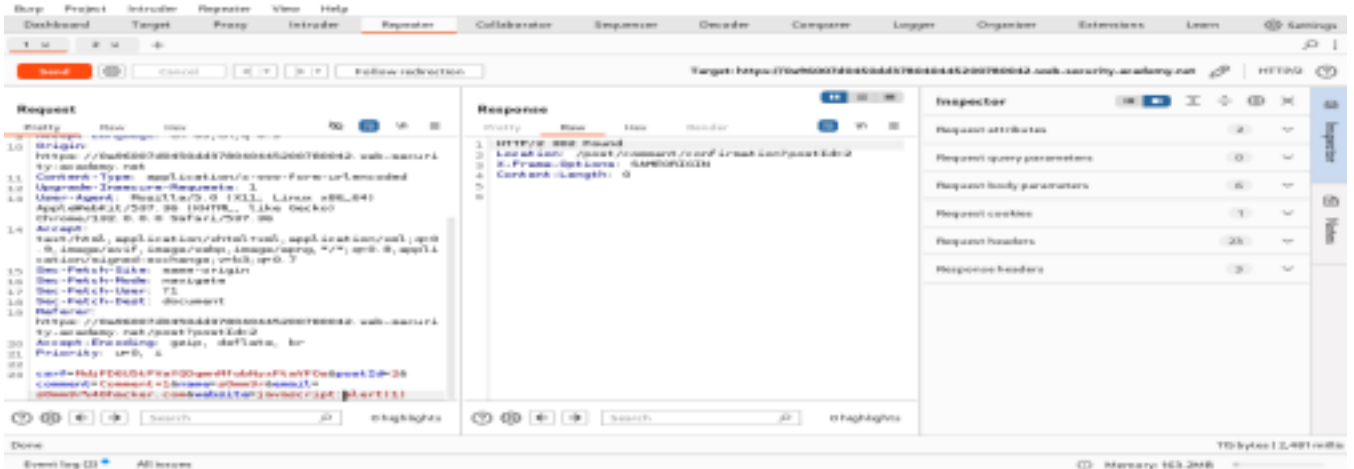
Passo 7 – Clique em Repeater na guia 1 e depois em Send



Passo 8 – Dentro do Repeater na guia 2 clique em Send.

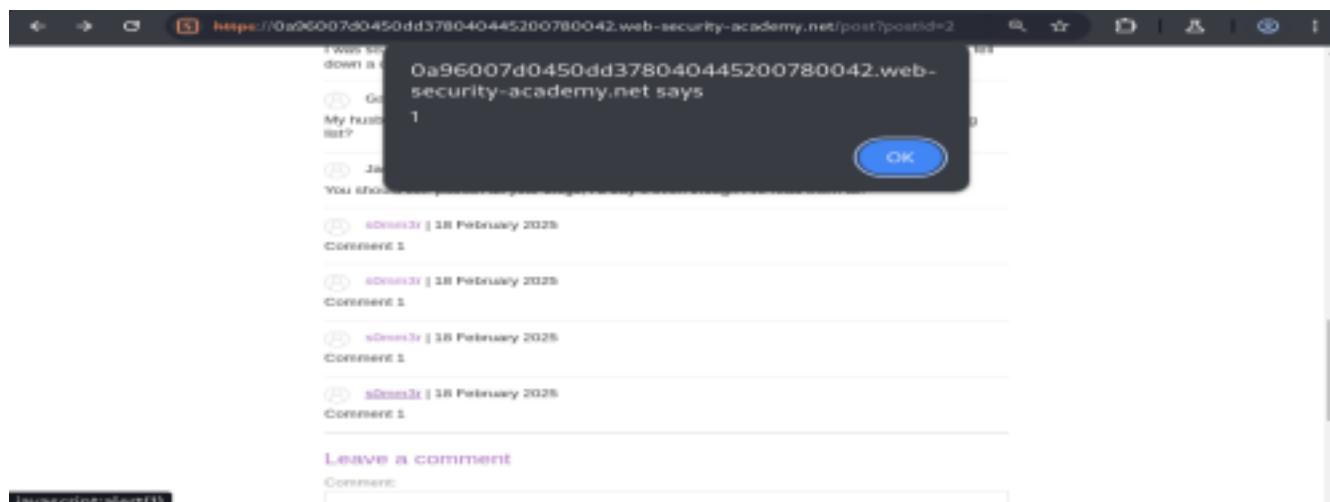


Passo 9 – Volte até a guia 1 e modifique no final o campo website para javascript:(alert(1)) um xss que irá gerar o alerta.

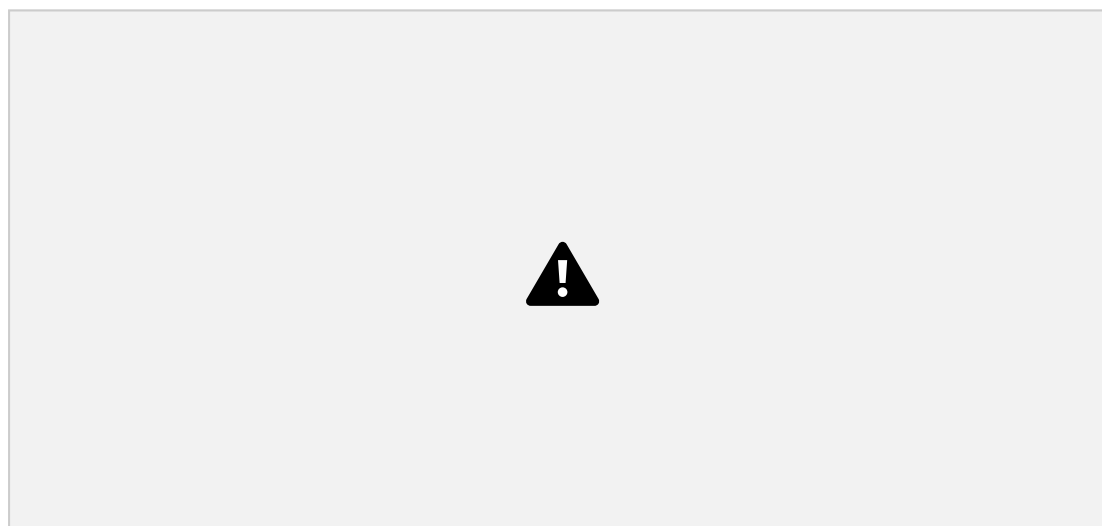


7

Passo 10 – A caixa de alerta aparece quando clica no nome s0mm3r.



Passo 11 – O laboratório está concluído



2.5. Atividade 3 – DOM XSS refletido.

Passo 1 - Vai aparecer o site: <https://0a2b00b1104f179de806e2ba30082001e.web-security-academy.net/> no campo search digite XSS e clique em Search.



Passo 2 – Vá até o Burp na Aba Proxy em Http History



Passo 3 – Depois vá até o Target no site clique na setinha para baixo clique em js e no searchresults.js



Passo 4 – Volte no campo de busca digite \"-alert(1)}// este comando xss gera um alerta e clique em Search



Passo 5 – Vai aparecer no campo depois da pesquisa a janela de alerta 1 .



Passo 6 – O laboratório está concluído



2.6. Atividade 4 – DOM XSS armazenado

Passo 1 - Vai aparecer o site: <https://0a1700b9045d89d8358c41f00c50029.web-security-academy.net/> clique em View Post.



Passo 2 – Vá na sessão de comentários e digite no comentário o comando xss quer irá gerar o alerta `<><img src=1 onerror=alert(1)>` e clique em post comment



Passo 3 – Aparecerá que seu comentário foi enviado.



Passo 4 – Depois clique em Back to blog.



Passo 5 – O laboratório está concluído



2.7. Análise dos Resultados:

Foi realizada a instalação do Burp Community no Kali Linux

Interpretação dos resultados:

- 1) Na Atividade 1 - Colocar um código HTML no campo de busca (search) para que a página web emita uma alerta, , quando o usuário acessa a URL enviada pelo invasor então o script é executado no browser do cliente, podendo executar algumas ações e recuperar os dados aos quais o usuário tenha acesso
- 2) Na Atividade 2 – Foi inserido no browser após o site o /abscd1234 mas apareceu que não foi encontrado, através de um comentário escrito em javascript em uma página web conseguimos inserir um alerta na página, ele recebe dados de uma fonte não confiável e inclui na resposta HTTP de forma insegura, ele pode enviar um script malicioso dentro do comentário, não precisa convencer o usuário a clicar em um link basta esperar que clique no seu comentário para executar as ações do script do atacante.

- 3) Na Atividade 3 – Utilizar o campo search com o XSS em busca de vulnerabilidade DOM para chamar uma função de alerta, os sites refletem dados da URL a

vulnerabilidade do lado do cliente, processando os dados da solicitação e ecoa os dados de string, javascript dentro do DOM como o campo de formulário para executar seu script malicioso.

- 4) Na Atividade 4 – Na sessão comentários digite um comando XSS para uma vulnerabilidade armazenada de DOM, os sites podem armazenar os dados no servidor, ele recebe estes dados de uma solicitação e os inclui numa resposta e processa os dados de forma insegura deixando armazenado o script malicioso.

2.8. Mitigação:

- 1) No laboratório 1 - Validação no campo de entrada do search para que não sejam escritos comandos que façam manipulação de XSS;
- 2) No laboratório 2 – Validação no campo de entrada dos comentários para que a manipulação não seja feita em comentários abertos a todos;
- 3) No laboratório 3 - Proteger a entrada de campos DOM reflexivo para que navegadores e scripts não manipulem este campo;
- 4) No laboratório 4 – Proteger a entrada de campos DOM armazenado para que os navegadores e scripts não manipulem este campo .

3. Considerações finais

Nesta sala, exploramos técnicas Pentest em Aplicações Web com o software Burp Suite Community para manipular algumas requisições, utilizando o Repeater para enviar requisições automaticamente, alguns comandos html para cross site scripting para manipular as pesquisas e comentários de páginas web, também utilizamos o browser para algumas requisições.

As principais lições aprendidas incluem:

- 1) A validação de entrada nos campos search e comentários é muito importante para que não haja nenhum tipo de manipulação
- 2) Verificar as páginas web para serem protegidas contra scripts maliciosos.
- 3) No campo de comentários verificar se não há links maliciosos.
- 4) Codificar os dados na saída para evitar que seja interpretada como conteúdo ativo, dependendo poderá exigir a aplicação de combinações de codificação.
- 5) Use cabeçalhos de resposta você pode utilizar cabeçalhos Content-Type e X-Content Type-Options para que os navegadores interpretem de maneira correta a solicitação.

- 6) Política de Segurança de Conteúdo pode ser utilizado para reduzir qualquer

vulnerabilidade XSS.

- 7) Codificar dados na saída deve ser aplicado antes que os dados de usuário sejam gravados em uma página, em HTML deve converter valores não listados em entidades HTML, em JavaScript os valores devem ser em formato unicode.
- 8) Usar listas brancas colocando apenas protocolos seguros e negar tudo que não estiver nesta lista branca.
- 9) Permitir HTML seguro através de filtrar tags de lista branca mas temos muitas diferenças entre os mecanismos de análise os navegadores e com a mutação de XSS esta abordagem é muito difícil de implantar e Javascript que executa filtragem e codificação no navegador, outras bibliotecas podem converter a marcação em HTML, porém as vulnerabilidades XSS continuam a evoluir e deve sempre atualizar estas bibliotecas contra novos ataques.
- 10) Evitar XSS do lado do cliente em Javascript deve utilizar um codificador HTML pois o Java não tem uma API para HTML, se a entrada estiver em JavaScript precisará de codificador para o Unicode para evitar o Cross-Site Scripting.

O Burp também é bem fácil de utilizar e tem recursos que deixam a manipulação do site ainda mais fácil como o Repeater e utilizar códigos XSS é muito simples.

4. Referências

Site: <https://portswigger.net/web-security/cross-site-scripting/contexts/lab-attribute-angle-brackets-html-encoded>

Site: <https://portswigger.net/web-security/cross-site-scripting/contexts/lab-href-attribute-double-quotes-html-encoded>

Site: <https://portswigger.net/web-security/cross-site-scripting/dom-based/lab-dom-xss-reflected>

Site: <https://portswigger.net/web-security/cross-site-scripting/dom-based/lab-dom-xss-stored>

Site: <https://portswigger.net/web-security/cross-site-scripting>

Site: <https://portswigger.net/web-security/cross-site-scripting/dom-based>

Site: <https://portswigger.net/web-security/cross-site-scripting/preventing>